

Async/Await

Promises, but readable

Agenda

1. Opening: same thing, two ways
2. Prior knowledge check
3. Core concepts
 - `async` — always a Promise
 - `await` — pause, not block
 - Error handling with try/catch
 - The `response.ok` trap
4. In-class assignment: Async JS pages
5. Reflection

Two ways to do the same thing

```
// Promises
fetch('/api/cars')
  .then(r => r.json())
  .then(envelope => renderCars(envelope.data))
  .catch(err => console.error(err));
```

```
// Async/Await
async function loadCars() {
  try {
    const response = await fetch('/api/cars');
    const envelope = await response.json();
    renderCars(envelope.data);
  } catch (err) {
    console.error(err);
  }
}
loadCars();
```

What prints — and in what order?

```
console.log('A');  
  
fetch('/api/cars')  
  .then(r => r.json())  
  .then(data => console.log('B', data));  
  
console.log('C');
```

Predict the output before running it.

Answer: A → C → B

```
A  
C  
B { meta: ..., data: [...] }
```

`fetch` is non-blocking. `C` prints before the response arrives.

async — always returns a Promise

```
async function greet() {  
  return 'hello';  
}  
  
greet(); // → Promise { 'hello' }
```

Every **async** function returns a Promise — even when you return a plain value.

await — pauses the function, not the program

```
async function loadCars() {  
  const response = await fetch('/api/cars'); // ← pause here  
  const envelope = await response.json();   // ← pause here  
  return envelope.data;                     // ← resume  
}
```

await suspends *this function* until the Promise resolves.

The rest of JavaScript keeps running.

Promise chain → Async/Await

Promise chain	Async/await
<code>.then(r => r.json())</code>	<code>await r.json()</code>
<code>.then(d => renderCars(d))</code>	<code>renderCars(d)</code>
<code>.catch(e => ...)</code>	<code>try { } catch (e) { }</code>

Same mechanism. Different syntax.

Error handling

```
async function loadCars() {
  try {
    const response = await fetch('/api/cars');
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }
    const envelope = await response.json();
    return envelope.data;
  } catch (error) {
    console.error('Failed:', error);
    return [];
  }
}
```

If an `await`-ed Promise rejects — it **throws**.

`try/catch` works exactly like synchronous code.

Watch out: `fetch` does not reject on 404

```
const response = await fetch('/api/cars/9999');  
// response.ok === false, but no error thrown!  
  
const envelope = await response.json();  
// envelope.error.status === 404
```

`fetch()` only rejects on **network failure**.

Always check `response.ok` — or inspect `envelope.error`.

await only works inside **async**

```
// ✗ SyntaxError
const data = await fetch('/api/cars');

// ✓ Works
async function init() {
  const data = await fetch('/api/cars');
}
```

Forgetting **async** gives a confusing `SyntaxError`.
The fix is always: wrap it in an **async** function.

In-class assignment

Build two async pages in classic-cars:

- `GET /async/cars` — fetch all cars, render a list
- `GET /async/cars/{id}` — fetch one car, render a detail card

The PHP shell is done together.

The JavaScript is yours to write.

Open `exercises.md` to get started.

Reflection

1. You have a `.then()` chain that works. What actually changes if you rewrite it with `async/await`?
2. Why does `fetch('/api/cars/9999')` not throw — and how do you handle it?
3. What is one thing you understand differently now compared to the start of this lesson?

Exit ticket

"Finish this sentence in your own words:

`await` pauses a function because..."